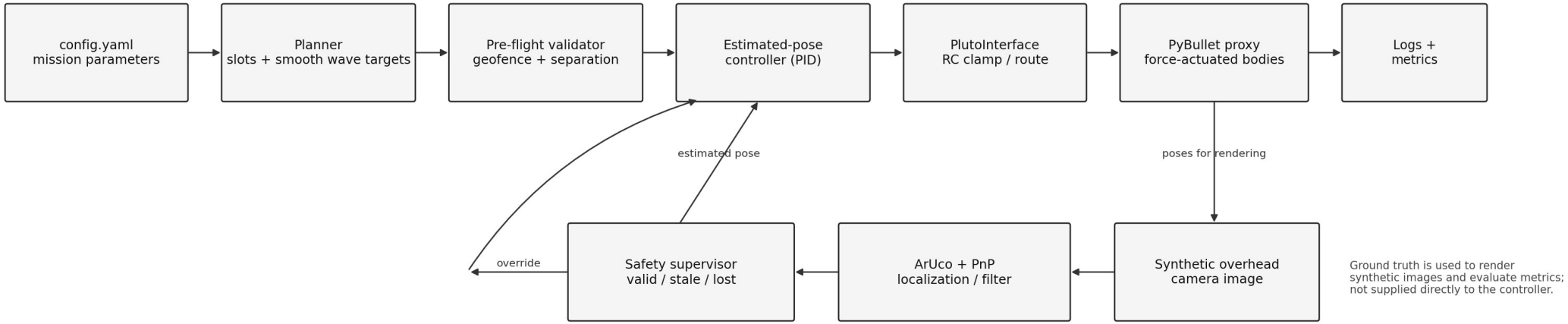
ENGINEERING DESIGN REPORT / GENERAL TRACK

SwarmSim-Lite: Synchronized Multi-Drone Formation and Wave Control

Inter-IIT Tech Meet 15.0 • Drona Aviation Mock PS • Roll No. 2501EE34 • Evidence cut-off: 05 November 2026

ABSTRACT
This work builds a four-agent swarm-control prototype around PyBullet, an OpenCV ArUco/PnP localization path, a generic line/circle trajectory planner, an estimated-pose feedback controller and a safety supervisor. The code includes N=10 pre-flight validation and logged fault drills. Two 23 s, three-cycle wave runs were executed, but both missed the Mock PS tracking and phase targets. The simulator is a force-actuated proxy rather than a calibrated rotor-level Pluto model; no physical flight is claimed.

01 / SYSTEM ARCHITECTURE



IMPLEMENTED DATA PATH

The planner creates line/circle slots and phased vertical-wave targets. The 20 ms validator checks the planned geometry against geofence and separation constraints. The synthetic overhead camera projects simulator poses into marker images; OpenCV detects ArUco IDs and estimates pose with PnP. A smoothed estimate feeds the controller and safety supervisor. RC commands pass through PlutoInterface to the PyBullet world. Logs and plots are generated for evaluation.

CONTROL / INFORMATION BOUNDARY

Ground truth is read by the camera renderer to create synthetic pixels and by the evaluation path for error calculation. The controller receives localization estimates, not direct simulator state. This is the correct separation for a vision-in-the-loop simulation; camera-rendering from truth is not the same as feeding truth to control.

ENGINEERING CHOICES

- PyBullet supplies time-stepped rigid-body integration and collision handling.
- YAML centralizes operating parameters and random seed.
- The planner uses generic N-based slots, a shared phase clock, Hungarian assignment utility and quintic transitions.
- ArUco IDs bind camera detections to drone indices; PnP estimates x, y, z and yaw.
- The cascaded controller uses estimated position/velocity, bounded integral action and RC clamps.
- Safety logic handles stale/lost estimates, separation and geofence events.

IMPORTANT MODEL BOUNDARY

The inspected simulator creates box proxies (mass 0.08 kg) and applies world-frame thrust, horizontal RC-derived force, drag and constant drift through PyBullet. Roll/pitch/yaw are updated as display attitude; they are not a rotor/moment-driven rigid-body flight model. The inspected implementation does not include calibrated Pluto inertia, motor dynamics, ground effect or downwash. PlutoInterface currently targets the simulator object, not a verified radio/firmware transport.

SCOPE NOTE The inspected mission runner performs a 5 s formation transition and then the 18 s three-cycle wave. It does not show the complete specified sequence of a separate 5 s slot hold, 3 s post-wave hold and controlled landing; this remains a compliance item.

02 / MATHEMATICAL ANALYSIS

A. QUADROTOR DYNAMICS AND UNDERACTUATION

$$\dot{p} = v$$
$$m\dot{v} = T R e_3 - m g e_3 + f_d$$
$$\dot{R} = R[\Omega]_\times \; ; \quad J\dot{\Omega} + \Omega \times (J\Omega) = \tau$$

Here p and v are world position and velocity; R is attitude; Ω is body angular velocity; m is mass; J is inertia; T is total thrust; τ is body torque; and f_d is disturbance/drag. A quadrotor has six configuration degrees of freedom but four independent rotor inputs (total thrust and three torques). It is underactuated: lateral acceleration is produced by tilting the thrust vector, not by an independent horizontal rotor input.

MODEL LIMIT The project has no Pluto URDF/MJCF or calibrated inertia/thrust constants in the inspected files. The equations above describe the standard vehicle model for analysis; they are not the equations actually integrated for the current box proxy. The proxy uses m=0.08 kg and thrust-to-weight ratio 3.0 from YAML.

B. WAVE KINEMATICS AND THRUST BOUND

$$z_i = 1 + A\sin(\omega t - \varphi_i)$$
$$\dot{z}_i = A\omega\cos(\omega t - \varphi_i) \; ; \quad \ddot{z}_i = -A\omega^2\sin(\omega t - \varphi_i)$$
$$|\dot{z}|_{max} = A\omega \; ; \quad |\ddot{z}|_{max} = A\omega^2$$

For A=0.20 m and P=6 s: ω=2π/P=1.0472 rad/s; peak vertical speed=0.2094 m/s; peak vertical acceleration=0.2193 m/s².

$$T_{max,nom} = m\,g \cdot TWR = 0.08 \cdot 9.81 \cdot 3 = 2.3544\ N$$
$$T_{avail,93\%} = 0.93\,T_{max,nom} = 2.1896\ N$$
$$\omega_{max} = \sqrt{[(0.93\,T_{max,nom}/m - g)/A]} = 9.37\ rad/s$$

This ideal thrust-only bound corresponds to a 0.671 s period. It assumes pure vertical sinusoidal acceleration and ignores controller bandwidth, latency, motor response and tilt coupling. It is not a measured tracking limit. For combined acceleration, $T_{req}=m\sqrt{[a_x^2+a_y^2+(g+a_z)^2]}$ must remain ≤0.93 Tmax.

C. FORMATION SEPARATION AND N=10 GEOMETRY

$$d_{ij}(t)=\sqrt{[(x_i-x_j)^2+(y_i-y_j)^2+(z_i-z_j)^2]}$$

For line phases φ_i,φ_j and $k=|i-j|$:

$$d_{ij}(t)=\sqrt{[(kd)^2+4A^2\sin^2(\Delta\varphi/2)\cos^2(\omega t-(\varphi_i+\varphi_j)/2)]}$$
$$\min_t d_{ij} = kd \; ; \quad \text{adjacent line drones: } d_{min}=d$$

With configured d=0.45 m, N=4 spans 1.35 m and the nominal minimum is 0.45 m.

$$\text{Circle: } \theta_i=2\pi i/N \; ; \quad \Delta\theta=\theta_i-\theta_j$$
$$d_{ij}(t)=2|\sin(\Delta\theta/2)|\sqrt{[R^2+A^2\cos^2(\omega t-(\theta_i+\theta_j)/2)]}$$
$$\text{nearest-neighbour } \min_t d_{ij} = 2R\sin(\pi/N)$$

With configured R=0.42 m, N=4 chord=0.5940 m. For N=10, chord=0.2596 m, below the 0.35 m rule. The minimum centre-point radius required for N=10 is R≥0.35/[2 sin(18°)]=0.5663 m. This leaves only 0.0337 m to the y geofence at ±0.60 m, before body size or uncertainty.

For the configured x-axis line, N=10 at d=0.45 m needs 9d=4.05 m, exceeding the 2.0 m x span. Even at the minimum permitted 0.35 m spacing it needs 3.15 m. Maximum count along x is 5 at d=0.45 m, or 6 if spacing is reduced to 0.35 m. For R=0.42 m, the largest N satisfying the chord-only bound is 7. Since the y geofence caps a centred circle at R≤0.60 m, the theoretical centre-point limit under separation alone is N=10; N=11 would fall below 0.35 m. N=10 therefore fits only near the boundary and with very little physical margin.

D. CAMERA GEOMETRY

$$D = 3 - z \; ; \quad X=(u-640)D/900 \; ; \quad Y=(360-v)D/900$$

At z=0.60 m (D=2.40 m), footprint=3.413×1.920 m; at z=1.40 m (D=1.60 m), footprint=2.276×1.280 m. One pixel laterally corresponds to D/f: 2.667 mm at z=0.60 m and 1.778 mm at z=1.40 m (same magnitude for y). For an 8 cm marker, s=fL/D and z=3−fL/s; $|\partial z/\partial s|=D^2/(fL)$=80.0 mm/px at z=0.60 m and 35.6 mm/px at z=1.40 m. Scale-based z is about 30×/20× more sensitive than lateral position.

E. CONTROL LINEARIZATION AND GAIN INTERPRETATION

$$\text{Near hover: } \ddot{x} \approx g\theta \; ; \quad \ddot{y} \approx -g\varphi \; ; \quad \ddot{z} \approx \delta T/m$$
$$v_d = v_{ref} + K_p e_p \; ; \quad a_{cmd} = a_{ref} + K_v(v_d - v_{est}) + I$$

For an ideal double-integrator with an outer proportional position loop and inner proportional velocity loop, the characteristic polynomial is $s^2+K_v s+K_vK_p$. Using YAML gains, x/y: Kp=1.8, Kv=1.2 gives $\omega_n=\sqrt{(2.16)}=1.470$ rad/s and $\zeta=K_v/(2\omega_n)=0.408$. z: Kp=3.0, Kv=1.8 gives $\omega_n=2.324$ rad/s and $\zeta=0.387$. These are idealized predictions only; filtering, saturation, integral action, RC mapping and the proxy plant alter the response.

STEP RESPONSE No verified predicted-versus-simulated step-response plot or rise/overshoot/settling-time dataset was present in the supplied evidence. This specific Mock PS math item remains unverified and must not be claimed complete.

03 / METHODOLOGY-ONLY ENGINEERING

The Mock PS explicitly allows these time-heavy topics to be explained without implementing code. The following are proposed methods, not claims of completed features unless separately identified.

1. KALMAN FILTER WITH LATENCY COMPENSATION

$$x=[px,py,pz,vx,vy,vz]^T \;;\; F=[[I,\Delta tI],[0,I]] \;;\; H=[I\ \ 0]$$
$$Q=q[[\Delta t^3 I/3,\ \Delta t^2 I/2],[\Delta t^2 I/2,\ \Delta t I]] \;;\; R=\mathrm{diag}(\sigma x^2,\sigma y^2,\sigma z^2)$$

Predict with $x^- = Fx$ and $P^- = FPF^T + Q$; update using the timestamped PnP position, innovation $y = z - Hx^-$ and Kalman gain $K = P^- H^T (HP^- H^T + R)^{-1}$. Keep a state/covariance and command buffer. When an image arrives 80 ms late, update at its capture timestamp, then replay stored commands to now. Account separately for the 40 ms command delay by propagating with the command that was actually active. Reject implausible innovations before update. The current implementation uses low-pass pose smoothing, not this delayed EKF.

2. YAW DRIFT AND FRAME TRANSFORMS

$$\psi \text{ drift} = 0.5^\circ/s \times 6\ s = 3^\circ \text{ per 6 s wave period}$$
$$a_{body} = Rz(-\psi)\, a_{world}$$

Without compensation, a desired world-x correction is applied partly along body-y as yaw drifts. Estimate yaw from marker orientation and rotate the desired world-frame acceleration into body coordinates before mapping it to roll/pitch. Re-zero the common heading only through a defined reference procedure; do not silently reset individual drone frames.

3. DOWNWASH MODEL AND VALIDATOR EXTENSION

$$r_h < \Delta z \tan(\alpha) \;\Rightarrow\; \text{lower drone is inside the downwash cone}$$
$$a_{dw} \approx k\, T_{upper} / (\Delta z^2 + r_h^2 + \varepsilon), \text{ directed downward}$$

Estimate cone half-angle α and coefficients k, ε from controlled two-drone vertical/horizontal sweeps in the simulator. Add a margin to the cone boundary and reject any sampled trajectory that enters the exclusion region. Calibrate against the selected physics model; do not treat this simple expression as a universal aerodynamic law. The inspected current proxy does not implement downwash.

4. THRUST AND TILT FEASIBILITY

$$T_{req} = m\sqrt{a_x^2 + a_y^2 + (g + a_z)^2} \le 0.93\, T_{max}$$
$$\beta = \mathrm{atan2}\left(\sqrt{a_x^2 + a_y^2},\ g + a_z\right) \le 15^\circ$$

Sample position, velocity and acceleration every 0.02 s. Check geofence, pairwise separation, thrust and tilt independently. Geometric feasibility means the path fits in space; dynamic/actuator feasibility means the vehicle can produce the required acceleration at depleted thrust. The current validator evidence covers geometric checks; dynamic limits are a proposed extension unless confirmed in source.

5. REMAINING FAULT DRILLS: F2 AND F4

F2 — Hide drone 3 marker for 1.2 s. Age each measurement from capture time: stale beyond 150 ms, lost beyond 300 ms. Hold on stale; on lost, land the affected drone and pause the rest of the swarm. The final status records a simulated F2 run with the marker hidden from 0.4 s to 1.6 s and stale/lost events. Its summary timestamps should be reconciled against capture-time age before being used as proof of exact 150/300 ms transitions. This is simulator evidence, not a physical test.

F4 — Stop commands to drone 0 for 1 s. Detect the firmware failsafe from vision-only motion/altitude signatures, corroborate with command-heartbeat timeout where available, pause the swarm and land safely. The project status reports a simulated watchdog case with drone 0 commanded to land after 1 s. The real MagisV2 firmware response must be verified with Drona; a software watchdog model is not proof of firmware behaviour.

6. SIMULATOR-NATIVE CAMERA

Replace OpenCV corner projection with a PyBullet camera render and an 8 cm DICT_4X4_50 texture fixed to each vehicle. Calibrate camera intrinsics/extrinsics, render at the prescribed pose and frame rate, and preserve the same ArUco detector/PnP interface. Compare synthetic-camera estimates against state at the exact capture timestamp. The current project uses a synthetic OpenCV-projected camera; a textured simulator-camera path is a proposed upgrade.

7. SCALING FROM N=4 TO N=10

$$\text{Pairwise separation checks: } O(N^2) = 45 \text{ pairs at } N=10$$
$$\text{Hungarian assignment: } O(N^3) = 1{,}000\text{-scale operations at } N=10$$

At N=10, raw algorithmic complexity is manageable. The practical limits are the fixed arena, marker pixel size, occlusion, camera field of view, communication airtime, packet delay and safety margins. The configured circle radius 0.42 m is insufficient for ten-drone 0.35 m chord separation; increasing it to at least 0.5663 m leaves little y-boundary margin. Ten-drone flight is not demonstrated.

IMPLEMENTATION STATUS

Topic	Status
Alpha-beta / smoothing	Implemented in localization path
Delayed EKF + replay	Methodology only
Yaw drift identification	Methodology only
Downwash model	Not in inspected proxy
Dynamic thrust/tilt validator	Proposed extension
F2 / F4	Reported simulated drills; no physical firmware validation
Native textured camera	Proposed extension

04 / RESULTS, LIMITATIONS AND NEXT-STAGE PIPELINE

MEASURED SIMULATOR OUTPUTS

Run	Duration	RMS target→estimate	Phase error	Min. separation	Localization RMS	Assessment
Line (static)	2 s	0.0143 m	N/A	0.4455 m	Not reported	Short-run tracking / separation pass
Circle (static)	2 s	0.0470 m	N/A	0.4037 m	Not reported	Short-run tracking / separation pass
Line + wave	23 s	0.1288 m	69.82° JSON / 104.93° status	0.4300 m	0.0215 m	FAIL tracking + phase; separation estimate passes
Circle + wave	23 s	0.1335 m	72.62° JSON / 77.49° status	0.3919 m	0.0345 m	FAIL tracking + phase; separation estimate passes

METRIC INTERPRETATION Tracking RMS, phase and minimum separation in the supplied metrics implementation are calculated from estimated positions, not simulator truth. Localization RMS compares estimate and truth at sample index; capture-time alignment needs verification because camera frames are delayed. Phase values conflict between JSON and FINAL_STATUS; both versions fail the 10° target. Reported wave RMS also exceeds the 0.10 m target.

VALIDATION AND LIMITATIONS

• 23 pytest tests are reported passing in the final verification record; this proves the tested functions, not full mission compliance.
• F1: drone 2 marker hidden for 0.4 s. F3: a single 1 m position jump is injected for drone 1; the configured 0.5 m jump gate is intended to reject it. Final status reports F1/F3 exercised in simulation and logged; no physical fault test is claimed.
• F2/F4 simulator drills are also reported. F2 transition timestamps need capture-age reconciliation; F4 watchdog reports drone 0 landing after a 1 s command timeout. Actual firmware response is unverified.
• N=10 validator artifacts exist. Analytically, line N=10 needs 4.05 m at configured d=0.45 m (or 3.15 m at 0.35 m), beyond the 2.0 m x-span. Circle N=10 at configured R=0.42 m has 0.2596 m chord, below 0.35 m.
• Control scheduling uses round(240/25)=10 physics steps, giving 24 Hz, not 25 Hz. Camera capture in that loop is likewise about 24 fps rather than the required 30 fps.
• The YAML sets vertical_rc_per_accel=650. For this proxy's thrust law, the inverse scale is 1000/(g·TWR)=33.98 RC/(m/s²); the configured scale is 19.13× larger and can saturate throttle.
• The normal mission-runner call does not consistently pass trajectory velocity/acceleration feed-forward. Lost localization lands that vehicle but the inspected safety code does not automatically pause the remaining swarm.
• The exact 5 s hold + 3 wave cycles + 3 s hold + land sequence is not shown in the inspected runner. No hardware run, ROS 2 bonus or ≤4 min demo video is evidenced.

CONCLUSION

The project establishes a modular vision-feedback and safety prototype with passing component tests and logged multi-drone runs. It does not yet meet wave tracking or phase targets, and its proxy dynamics and command mapping need correction before claiming flight readiness. The next step is to calibrate the plant, close the control/timing gaps and rerun the same logged acceptance suite.

RECOMMENDED PRACTICAL TOOLCHAIN

Physics: retain PyBullet, but use gym-pybullet-drones as a reference/upgrade path for URDF-based multirotor parameters and configurable drag, ground-effect and downwash models. Keep our own planner, validator, localization, PID and safety logic; do not adopt its ready-made controller. Its stock vehicle is not Pluto, so identify/replace parameters before treating it as a Pluto model. [1]

Integration: Ubuntu 22.04 + ROS 2 Humble (preferably WSL2 for development if device/GUI support is adequate), plus Drona's current Pluto ROS 2/Python interface. Drona's current software page advertises Humble, while the public ROS 2 repository README still describes Foxy setup; pin versions and verify message/API compatibility before integration. [2,3]

Perception / observability: OpenCV ArUco + cv_bridge; RViz2 for poses, targets, trails and geofence; rosbag2 for timestamped record/replay. Use pytest, NumPy, pandas and Matplotlib for repeatable regression and metrics. [4,5]



GATED SIM-TO-HARDWARE PIPELINE

Model identification → unit tests → single-drone SITL → N=4 + camera-in-the-loop → fault injection / multi-seed regression → ROS 2 topic and bag replay → propellers-off bench tests (RC mapping, arm/disarm, timeout, E-stop) → supervised 1-drone flight → static N=4 → line/circle waves. Each gate must archive configuration, code version, seed, logs and metrics; a simulation pass is not a hardware pass.

ACCEPTANCE GATES

Tracking RMS ≤0.10 m; phase error ≤10°; estimated/true minimum separation ≥0.35 m; no geofence breach; 25 Hz control and ≥240 Hz physics measured (not merely configured); fault responses verified. Physical flight begins only after safe bench and supervised single-drone gates.

REFERENCES

[M] Drona Aviation, "SwarmSim-Lite" Mock PS, pp. 1–6 (requirements and thresholds). [1] gym-pybullet-drones, project and BaseAviary documentation: github.com/learnsyslab/gym-pybullet-drones. [2] Drona Aviation software suite: dronaaviation.com/software. [3] DronaAviation/pluto_ros2_package README (Foxy setup): github.com/DronaAviation/pluto_ros2_package. [4] ROS 2 rosbag2: github.com/ros2/rosbag2. [5] ROS 2 RViz2 package index: index.ros.org/p/rviz2. [6] ROS 2 Humble Ubuntu installation notes (Jammy 22.04): docs.ros.org/en/humble. Accessed 05 Nov 2026.